

AA4. Diseño de diagrama de clases 2

Autor: Pol Valle

Año: 2026

Índice

Contenido

Introducción	3
Evolución del Programa	3
Diagrama de clases	4
Decisiones de diseño	6
Implementación en Java	6
Funcionalidades del menú	7
Comentarios sobre el desarrollo	7
Mayor dificultad	7
No completado	7
Mejoras posibles	7
Video	8
Autoevaluación	8

Introducción

En esta actividad se amplía el el programa de consola de WoodShoops de la actividad AA3 añadiendo gestión de clientes y de ventas.

Sobre el modelo de productos, proveedores y tiendas existente, se ha incorporado jerarquía de clientes (anónimo, profesión y WoodFriends, la emisión de tickets con sus líneas de detalle y el listado y resúmenes asociados

Evolución del Programa

Analizando el enunciado, las clases nuevas que se han añadido respecto a la AA3 son:

- **Cliente(Clase abstracta)**: con dos subclases, ClienteProfesional y CLienteWoodFriend.

El cliente anónimo de se modela con una clase propia, sino que se representa con null en el ticket, ya que no hay que almacenar ningun dato en el.

- **Ticket**: Almacena el número de fecha, cliente (puede ser null) y la lista de líneas de detalle
- **LineaTicket**: asocia un ProductoTienda, una cantidad y un precio unitario congelado en el momento de la venta.

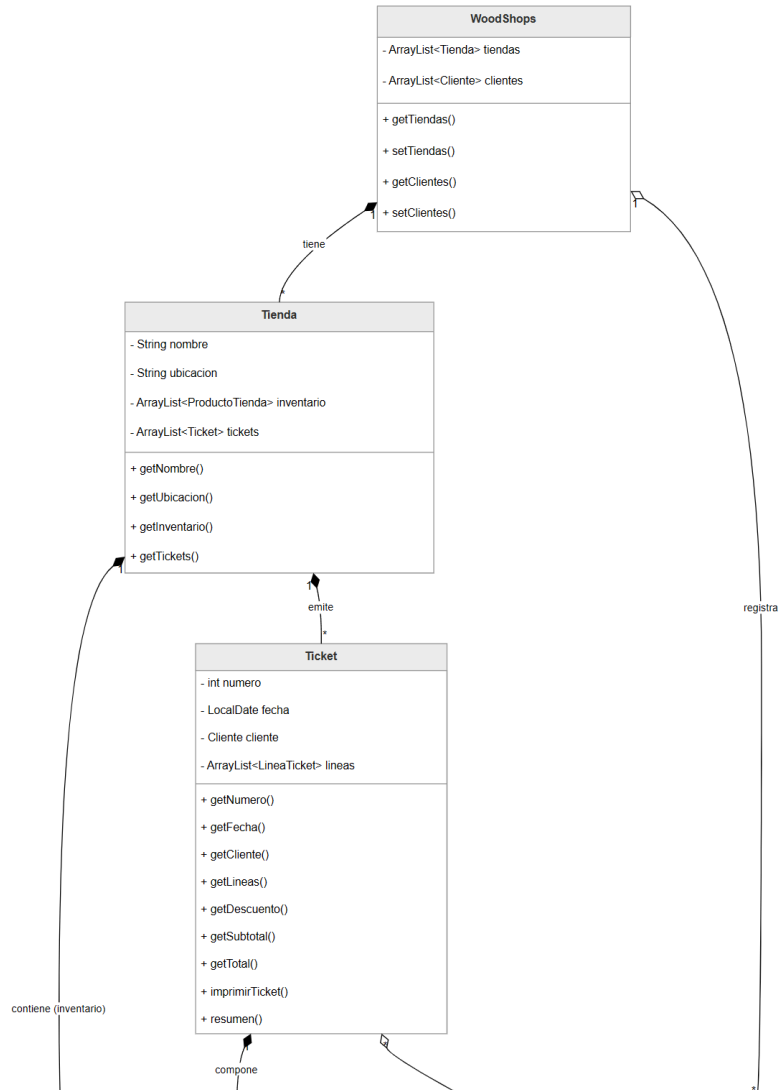
En la clase principal MainWoodShops se han añadido las opciones de menú correspondientes a las funcionalidades requeridas:

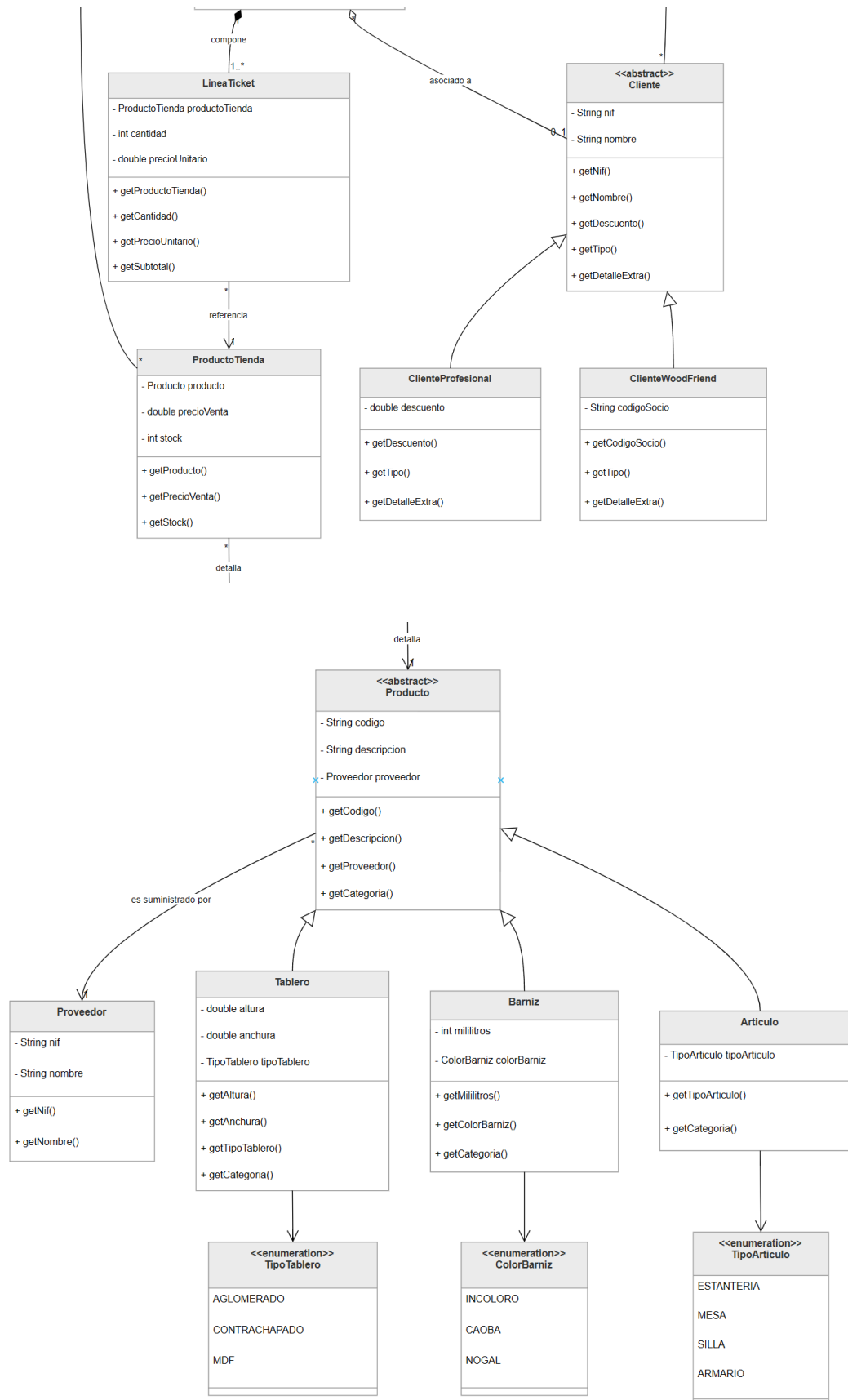
- **anadirCliente()**: Alta de cliente Profesional o WoodFriend, validando que el NIF no este ya registrado.
- **listarClientes()**: listado de todos los clientes con su tipo y detalle extra (descuento o código de socio).
- **anadirTicket()**: Creación de un ticket de venta seleccionando tienda, cliente productos y cantidades. Descuenta el stock y muestra el ticket completo con descuento aplicado
- **resumenTickets()** listado resumen de los tickets de una tienda filtrando por un rango de fechas.

Se mantiene la carga inicial de datos, que crea proveedores, productos, tiendas con inventario, los tres clientes de ejemplo (dos profesionales y un WoodFriend) y un ticket inicial de demostración con descuento profesional aplicado.

Diagrama de clases

El diagrama de





Decisiones de diseño

Cliente abstracta. Permite tratar todos los clientes de forma polimórfica en Ticket sin instanceof.

Cliente anónimo como null. El enunciado indica que del anónimo no se almacena ningún dato; crear una clase específica añadiría complejidad innecesaria.

Cliente como agregación (no composición). Tiene identidad propia (NIF, nombre) y existe al margen de la empresa. Una composición dejaría referencias colgantes en los tickets si se destruye WoodShops.

LineaTicket apunta a ProductoTienda: Necesita conocer el precio de la tienda concreta donde se hizo la venta, no solo el producto.

Precio unitario congelado en LineaTicket: Si el precio de la tienda cambia, los tickets históricos no se ven afectados.

Polimorfismo con getCategory() y getDetalleExtra(): Evita instanceof en MainWoodShops y cumple el principio abierto/cerrado.

Enumeraciones. Type-safe en compilación, menús dinámicos con values() y extensibles sin tocar la UI.

Tickets propios de cada Tienda. Las ventas las realiza la tienda y los resúmenes se piden por tienda.

Implementación en Java

Prácticas aplicadas:

- Validación de duplicados (existeCodigoEnTienda, existeNifCliente).
- Validación de rangos en descuentos e índices de enums.
- Captura de excepciones (InputMismatchException, DateTimeParseException).
- Menús genéricos sobre enums con leerEnum<E extends Enum<E>>.

- Pausa y limpieza de consola entre opciones (pausarYLimpiar) para mejorar legibilidad.
- Javadoc completo en todas las clases y métodos públicos.

Funcionalidades del menú

Comentarios sobre el desarrollo

Mayor dificultad

Decidir qué clases referencian ProductoTienda y cuáles Producto, manteniendo el contexto de la tienda.

Modelar el cliente anónimo como null tras descartar crear una clase específica.

Congelar el precio en LineaTicket para que los tickets históricos no varíen.

Eliminar instanceof introduciendo getCategory() y getDetalleExtra().

Gestionar nextInt() / nextLine() y capturar InputMismatchException.

No completado

Las "ofertas especiales" de WoodFriend no se han modelado al no concretarlas el enunciado.

No hay persistencia: los datos se pierden al cerrar el programa.

Mejoras posibles

Bases de datos real

Interfaz gráfica para manipulación de datos

API REST

Video

<https://youtu.be/Te2GvtgBTDY>

Autoevaluación

Criterio	Justificación	Puntos
Diagramas de clases	Diagramas completo con jerarquías, con decisiones de diseño justificadas	7.5/10
Generación de código	Herencia. Polimorfismo, enums, validaciones	7.5/10